

Index in GROUP BY Clause in SQL Server

In this article, I am going to discuss how to use **Index in Group by Clause in SQL Server** as well as I am also going to discuss Covering Query in SQL Server with examples. Please read our previous article, where we discussed [SQL Server Unique Index](#) with examples.

Let's first discuss the Algorithm used by SQL Server to group the data.

The SQL Server database uses two different groups by algorithms.

1. The first one is the hash algorithm which will aggregate the input records into a temporary hash table. Once all the input records are processed, then the hash table is returned as the result.
2. The second algorithm is the sort/group algorithm which will first sort all the input data by the grouping key so that the rows of each group follow each other in immediate succession. Afterward, the database just needs to aggregate them.

In general, both algorithms need to materialize an intermediate state, so they are not executed in a pipelined manner. Nevertheless, the sort/group algorithm can use an index to avoid the sort operation, thus enabling a pipelined group by.

The index in GROUP BY Clause in SQL Server

Creating an Index on the column that is used in the GROUP BY clause can greatly improve the performance of the query in SQL Server. **Let's understand this with an example.** We are going to use the following ProductSales table in this demo

ProductSalesID	OrderID	ProductID	QunatitSold
1	1001	501	3
2	1001	502	2
3	1001	503	4
4	1002	501	1
5	1002	502	2
6	1003	503	2
7	1003	501	3
8	1004	502	4
9	1004	503	4
10	1001	501	3
11	1001	502	2
12	1001	503	4
13	1002	501	1
14	1002	502	2
15	1003	503	2
16	1003	501	3
17	1004	502	4
18	1004	503	4

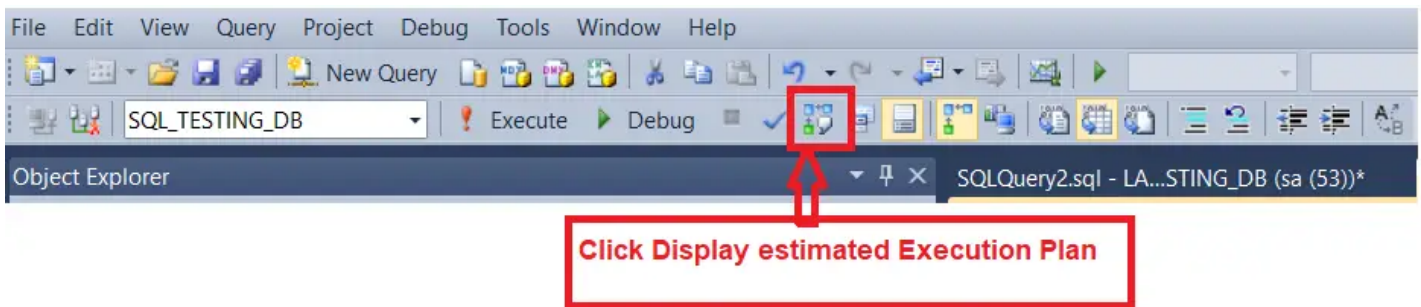
Please use the below SQL Script to create and populate the ProductSales table with test data.

```
-- Create the ProductSales table
CREATE TABLE ProductSales
(
    ProductSalesID INT,
    OrderID INT,
    ProductID INT,
    QunatitSold INT
)
GO
-- Populate the ProductSakes table with some test data
INSERT INTO ProductSales VALUES(1, 1001, 501, 3)
INSERT INTO ProductSales VALUES(2, 1001, 502, 2)
INSERT INTO ProductSales VALUES(3, 1001, 503, 4)
INSERT INTO ProductSales VALUES(4, 1002, 501, 1)
INSERT INTO ProductSales VALUES(5, 1002, 502, 2)
INSERT INTO ProductSales VALUES(6, 1003, 503, 2)
INSERT INTO ProductSales VALUES(7, 1003, 501, 3)
INSERT INTO ProductSales VALUES(8, 1004, 502, 4)
INSERT INTO ProductSales VALUES(9, 1004, 503, 4)
INSERT INTO ProductSales VALUES(10, 1001, 501, 3)
INSERT INTO ProductSales VALUES(11, 1001, 502, 2)
INSERT INTO ProductSales VALUES(12, 1001, 503, 4)
INSERT INTO ProductSales VALUES(13, 1002, 501, 1)
INSERT INTO ProductSales VALUES(14, 1002, 502, 2)
INSERT INTO ProductSales VALUES(15, 1003, 503, 2)
INSERT INTO ProductSales VALUES(16, 1003, 501, 3)
INSERT INTO ProductSales VALUES(17, 1004, 502, 4)
INSERT INTO ProductSales VALUES(18, 1004, 503, 4)
```

Generally, we use a GROUP BY clause to group the records and their aggregate values. For example, counting the number of product sales with the same ProductId. To process a query with a GROUP BY clause, the database will often sort the results on the columns included in the GROUP BY clause.

How to see the query Execution Plan in SQL Server

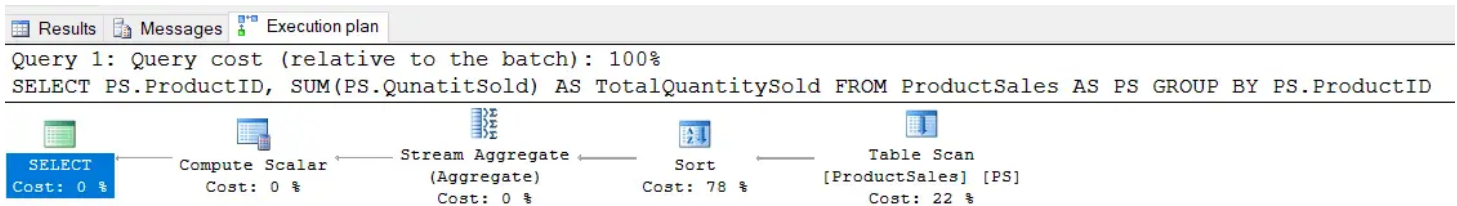
To see the query execution plan, click on the Estimated Query Execution plan option as shown below. We will discuss the Query Execution Plan in SQL Server in a later article. For now, just understand how we can see the query execution plan in SQL Server.



Please consider the following query

```
SELECT PS.ProductID,
       SUM(PS.QunatitSold) AS TotalQuantitySold
FROM   ProductSales AS PS
GROUP BY PS.ProductID
```

In the above query, we are doing a SUM on the QuantitySold column and grouping it by the ProductId column. The business requirement is to get the total quantity sold for each product. **When we run the above query, we get the following execution plan**



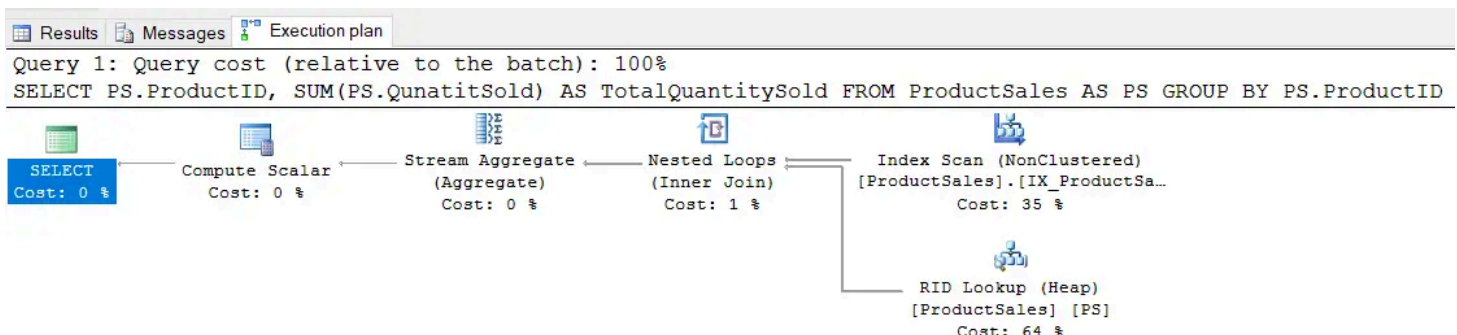
As you can see from the above image, it uses a Table scan and we know the table scan is bad for performance as it needs to scan each and every element present in the table. **Let's try to improve the query by creating an index on the ProductId column and then see the cost of the query.**

CREATE NONCLUSTERED INDEX IX_ProductSales_ProductID ON ProductSales(ProductID)

Now let's execute the same query.

```
SELECT PS.ProductID,
       SUM(PS.QunatitSold) AS TotalQuantitySold
FROM   ProductSales AS PS
GROUP BY PS.ProductID
```

Once we execute the above query, now it gives us the below query execution plan.



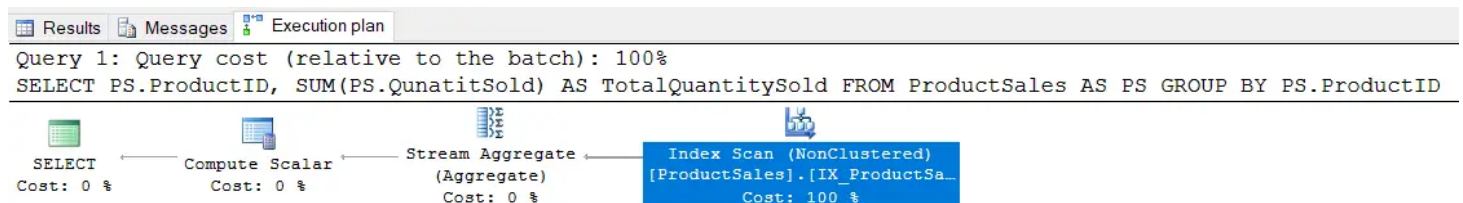
As you can see from the above query execution plan image, now it uses an Index Scan instead of a Table scan and you can also see that it used the IX_ProductSales_ProductID non-clustered index. But along with it also used nested loops and Rid Lookup and we can improve this also. **Now let's create an index on both ProductID and QuantitySold columns as shown below.**

```
CREATE NONCLUSTERED INDEX IX_ProductSales_ProductID_QunatitSold ON ProductSales(ProductID, QunatitSold)
```

Now again execute the Group By query to get the data as shown below.

```
SELECT PS.ProductID,  
       SUM(P.S.QunatitSold) AS TotalQuantitySold  
FROM ProductSales AS PS  
GROUP BY PS.ProductID
```

It gives us the below execution plan.



When you mouse over to the Index Scan, you will see that it uses the newly created index i.e. **IX_ProductSales_ProductID_QunatitSold** instead of the index **IX_ProductSales_ProductID**. So this proves that the SQL Server chooses the best query plan execution based on the available indexes.

What is a Covering query?

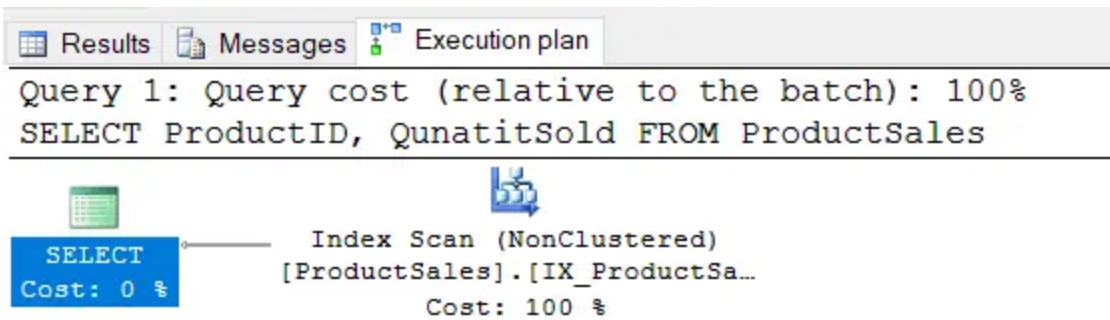
If all the columns that we have requested in the select clause of the query are present in the index, then there is no need to look up the table again. The requested column data can simply be returned from the index. This is called Covering Query.

Please consider the following example.

```
SELECT ProductID, QunatitSold FROM ProductSales
```

From the above select query, the requested ProductID and QuantitySold column can simply be returned from the Index **IX_ProductSales_ProductID_QunatitSold** as this index contains the ProductID and QunatitSold column data.

The above query gives us the below Execution plan.



In the next article, I will discuss the [Advantages and Disadvantages of using Indexes in SQL Server](#) with examples. Here, in this article, I try to explain how to use the **Index in Group by Clause in SQL Server** step by step with some examples. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.